I.J. Information Engineering and Electronic Business, **2026, 4, 176-193**
Published Online on August 8, 2026 by MECS Press (http://www.mecs-press.org/)
DOI: 10.5815/ijieeb.2026.04.12

Enhanced Educational Recommendations via Feedback-Refined Learning-to-Rank with XGBoost

Sushilkumar Chavhan*
Department of Information Technology, Yeshwantrao Chavan College of Engineering, Nagpur, India
Email: schavhansushil@gmail.com
ORCID iD: https://orcid.org/0000-0002-6468-8196
*Corresponding author

Ujwalla Gawande
Department of Information Technology, Yeshwantrao Chavan College of Engineering, Nagpur, India
Email: ujwallgawande@yahoo.co.in
ORCID iD: https://orcid.org/0000-0002-6956-6833

Sachin Jain
Department of CS, Oklahoma State University, United States
Email: sachin.jain@okstate.edu
ORCID iD: https://orcid.org/0000-0002-1343-2502

Nikhil Mangrulkar
Symbiosis Institute of Technology, Nagpur Campus, Symbiosis International (Deemed University), Pune, India
Email: mangrulkar.nikhil@gmail.com
ORCID iD: https://orcid.org/0000-0001-5190-9722

Rajesh Dharmik
Department of Information Technology, Yeshwantrao Chavan College of Engineering, Nagpur, India
Email: raj_dharmik@yahoo.com
ORCID iD: https://orcid.org/0000-0002-0334-711X

Devendra Shahare
Department of Mechanical Engineering, Yeshwantrao Chavan College of Engineering, Nagpur, India
Email: devshahare0501@gmail.com
ORCID iD: https://orcid.org/0000-0002-9955-6898

Received: October 10, 2025; Revised: November 11, 2025; Accepted: December 19, 2025; Published: August 8, 2026

Abstract: A system that is personalized and capable of automatically suggesting appropriate courses based on a user's particular questions and interests provides customized recommendations. The system employs an LTR model relying on XGBoost to learn relationships between the queries and the courses. Dynamic ranking feature refinement enhances ranking, and a feedback loop constructs incrementally improving the recommendations by applying relevant courses to update the model. The scraped educational sites are the foundation of the dataset, where there are granular course descriptions as well as the interaction logs. Evaluation results indicate that the system is able to produce high ranking outcomes as evidenced by an NDCG value of 0.85 and high values for MRR. The system is able to produce low query processing latency, making it possible for real-time responsiveness. User feedback analysis following system retraining indicated a 90% increase in user satisfaction. The suggested framework is dynamic and provides personalized recommendations for courses in various learning environments.

Index Terms: Personalized Recommendation System, Learning-to-Rank, XGBoost, Course Ranking, User Feedback, Educational Platforms.

This work is open access and licensed under the Creative Commons CC BY 4.0 License.

1. Introduction

The rapid growth of online learning platforms has created both significant opportunities and new challenges in delivering personalized educational content. Learners currently have access to thousands of courses on various subjects from websites such as Coursera, edX, and Udemy. On the other hand, this plethora of courses makes it hard for learners to choose the ones that best suit their interests and aspirations. Here, personalized recommendation systems are emerging as key tools for improving learning experiences and leading students to appropriate content [1].

These systems work with a view to providing course recommendations based on user profiles, previous interactions, and course information [2-3]. Creating such systems within educational contexts has its challenges. The most popular of these is the "cold start" issue where the system does not have sufficient information regarding new users or new courses upon which it can provide reliable recommendations [2]. Additionally, learners often change their preferences over time, making it necessary for the system to adapt continuously to remain relevant.

Classic approaches such as collaborative filtering and content-based filtering have been extensively utilized in course recommendation, yet both of them are subject to limitations. Collaborative filtering is prone to low performance when user-item interaction data is not dense [4], whereas content-based filtering is too restrictive and tends to recommend similar courses due to static features, while neglecting the changing interest of users [5,6].

To cater to these problems, researchers have begun employing Learning-to-Rank (LTR) models that are more appropriate for ranking tasks where the objective is not only to identify relevant items but also to rank them in order of importance or utility [7,8]. Of these, XGBoost has been a top contender because it is efficient, scalable, and can learn from intricate feature sets [9]. When integrated with LTR methods, XGBoost can correctly represent user-user, query-query, and course-course relationships to enhance the ranking of recommendations.

Notably, new developments highlight the critical importance of feedback for enhancing recommendation quality. Beyond the usage of historical data alone, systems can also be trained based on continued user behavior (e.g., clicks, course completion, ratings) in order to dynamically refresh recommendations [10, 11]. Such a feedback loop renders the recommendation engine more responsive and better able to cater to users in actual education contexts.

In this paper, we present a feedback-driven recommendation framework based on Learning-to-Rank with XGBoost, tailored for large-scale online education platforms. Our approach aims to deliver accurate, adaptive, and real-time course suggestions, helping learners find the right content at the right time.

This work is focused on

- How can Learning-to-Rank methods using XGBoost be applied to improve the ranking quality of course recommendations in e-learning platforms?
- What impact does a real-time feedback mechanism have on the personalization and adaptability of the recommendation system?
- How effective is the proposed system in terms of standard ranking metrics such as NDCG and MRR, compared to traditional recommendation techniques?

The paper is organized as follows: Section II reviews prior related research and provides the motivation for the current study. Section III provides a detailed overview of the proposed methodology. Section IV outlines the detailed design and experimentation of the proposed model and results are presented in Section V, followed by the conclusion and future scope.

2. Related Work

Recommendation systems have undergone considerable development over the past two decades, evolving from basic rule-based filtering to more advanced models incorporating machine learning and ranking algorithms. The success of any recommendation system largely hinges on its ability to accurately capture user preferences and rank relevant content in a way that aligns with individual needs.

Traditional approaches such as collaborative filtering and content-based filtering have been extensively used in many domains [15]. Collaborative filtering relies on past user-item interactions but is affected by data sparsity and the cold-start problem, precluding its utility for new users or courses [4, 5, 19]. Techniques of content-based filtering, though useful in making use of pre-defined item properties, tend to be unsatisfactory in responding to users' changing tastes. This can lead to redundant, too-specific recommendations that do not capture evolving learning settings [6]. Hybrid strategies, including those surveyed by Burke [12], try to leverage the strengths of various recommendation techniques. Unfortunately, they tend to demand heavy manual tuning and lack the scalability for real-time operations in dynamic settings.

To meet these challenges, Learning-to-Rank (LTR) models have been proposed as a stronger replacement. Tailored to ranking tasks, LTR models can learn ideal item orderings from annotated datasets that reflect user relevance judgments [7]. Of these approaches, XGBoost has proven to be highly effective because of its scalability, efficiency in computations, and capacity to model complex feature interactions between course metadata and user queries [9]. The

incorporation of XGBoost as a component of an LTR pipeline facilitates end-to-end optimization of ranking scores like the Normalized Discounted Cumulative Gain (NDCG), thus improving the accuracy of personalized suggestions [24].

The advantages of LTR-based methods have been shown in the earlier research. Qin et al. [8] and Joachims [13] showed great performance improvement in terms of ranking with pairwise and listwise ranking methods. Zhang et al. [10] further highlighted the importance of using user feedback to improve the learning process by exhibiting important improvements in real-time responsiveness and personalization through adaptive model updates [18].

Recent advances in recommendation systems have increasingly focused on deep learning–based models that learn complex user–item representations [14]. Survey studies highlight how neural architectures extend traditional collaborative filtering by modeling nonlinear interactions and rich contextual features [21]. Neural recommender models have demonstrated strong performance improvements in offline evaluations, particularly in dense interaction environments [20]. More recent research has explored transformer-based and sequential recommendation models, which leverage self-attention mechanisms to capture long-term dependencies in user behavior [22]. Such models have shown promising results in sequential ranking tasks and personalized content delivery [23]. However, these approaches often require large-scale interaction data, extensive hyperparameter tuning, and significant computational resources, which may limit their applicability in latency-sensitive or resource-constrained environments. In spite of these developments, most current systems are still lacking in dynamically refreshing rankings based on user activity. This creates a gap between theoretical gains and actual performance, particularly in high-speed education platforms. To close this gap, this work presents a feedback-driven course recommendation system based on LTR with XGBoost. The suggested system seeks to provide scalable, real-time, and extremely personalized course recommendations, effectively mapping recommendations to users' changing learning objectives.

3. Proposed Methodology

The following Personalized Course Recommendation System relies on a Learning-to-Rank (LTR) approach combined with the XGBoost algorithm to provide adaptive and context-aware course recommendations. The system design is divided into five dependent phases (as depicted in Fig. 1), which are intended to overcome various limitations of conventional recommendation methods and maximize the overall accuracy, scalability, and responsiveness of the recommendation process.

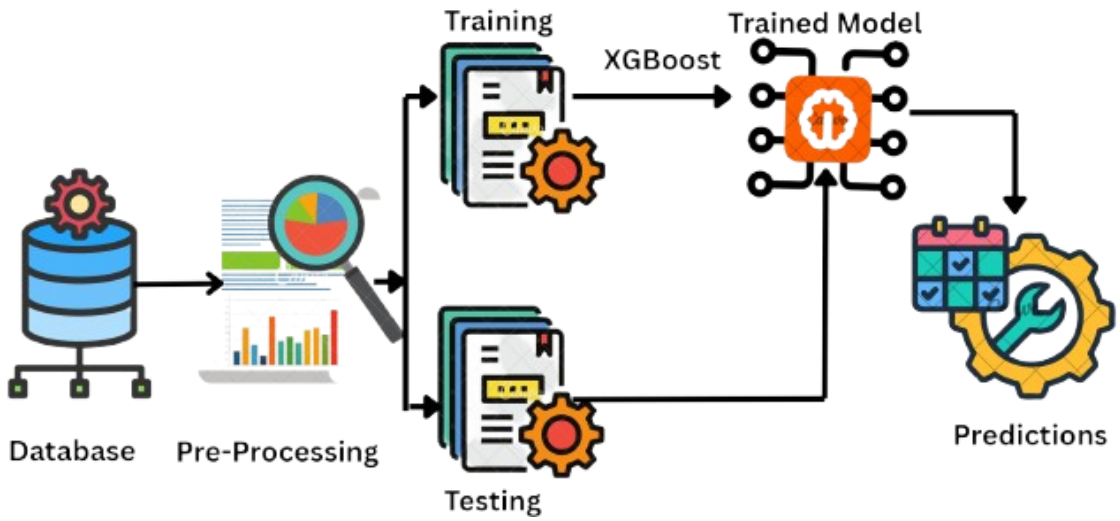


Fig. 1. Proposed Methodology.

3.1. Dataset Description

To support the development and testing of the Learning-to-Rank–based course recommendation system suggested, a large multi-source dataset was assembled by compiling data from three popular Massive Open Online Course (MOOC) websites: Coursera, Udemy, and edX. The reason for selecting these sites is that they have large user bases all over the world, a wide range of academic and professional topics, and varied teaching approaches. This selection ensures that the resulting dataset represents a broad range of learner profiles, content models, and interaction patterns offering a solid basis for developing a generalized and scalable e-learning recommendation system.

Data was acquired through a combination of web scraping (using platform-compliant crawling scripts) and synthetic user interaction simulation for personalization testing. The dataset includes a comprehensive set of 15 attributes per course, capturing both static metadata and dynamic engagement metrics (Table 1). Static attributes include course title, description, category, instructor details, difficulty level, duration, and pricing model. Dynamic attributes

include user ratings, reviews, enrollment counts, click-through rates (CTR), completion rates, and feedback scores. The raw dataset comprised 2.3 million records in total 1,000,000 from Coursera, 900,000 from Udemy, and 400,000 from edX. These records span 12 major subject categories (e.g., Computer Science, Business, Data Science, Health Sciences) and three difficulty levels (Beginner, Intermediate, Advanced). This breadth ensures adequate representation of both popular and niche courses. A detailed summary is provided in Table 1.

Table 1. Summary of Dataset Sources and Composition.

Source Platform	Number of Attributes	Number of Records	Description of Data
Coursera	15	1,000,000	Course metadata, instructor profiles, learner reviews, enrollment statistics, ratings
Udemy	15	900,000	Course metadata, instructor details, pricing models, learner interactions, ratings
edX	15	400,000	Course details, categories, difficulty levels, user engagement metrics
Total	—	**2,300,000**	Combined dataset covering diverse subjects, difficulty levels, and learner behaviors

Access to large-scale, fine-grained user interaction logs from commercial MOOC platforms is typically restricted due to privacy, ethical, and licensing constraints. To enable controlled evaluation of personalization and feedback mechanisms, synthetic user interaction data was therefore incorporated in this study.

The synthetic interactions were generated to closely mirror realistic learner behavior by following statistical distributions observed in prior educational recommender studies, including click-through rates, enrollment likelihood, completion patterns, and rating tendencies. These distributions were calibrated using publicly reported engagement statistics and empirical patterns commonly reported in large-scale e-learning platforms.

Importantly, synthetic data was used exclusively to simulate feedback dynamics and personalization effects, while course metadata and structural attributes were derived from real-world MOOC platforms. This design ensures that the core content representation remains realistic, while allowing systematic evaluation of the feedback-driven Learning-to-Rank mechanism.

We acknowledge that synthetic user interaction data may not capture the full diversity and complexity of real learner behavior and may introduce bias. This limitation is explicitly recognized, and future work will focus on validating the proposed framework using real user interaction logs or live user studies to further assess generalizability and robustness.

3.2. Data Cleaning and Preprocessing

In the data preprocessing stage, raw records were cleaned by removing duplicates, imputing missing values, and correcting inconsistencies. Numerical attributes such as ratings and engagement rates were normalized to the range [0,1], and categorical variables (e.g., course categories and difficulty levels) were label-encoded to make them compatible with tree-based models like XGBoost.

To process textual data, we applied natural language processing techniques including tokenization, lemmatization, stop-word removal, and vectorization using Term Frequency–Inverse Document Frequency (TF-IDF). The TF-IDF score for a term t in a document d, within the corpus DD, is computed as:

$$TF - IDF(t,d) = TF(t,d) \times \log\left(\frac{|D|}{1 + |\{d' \in D : t \in d^i\}|}\right) \quad (1)$$

where:
TF(t,d) is the term frequency of term t in document d,
|D| is the total number of documents,
$\{d' \in D : t \in d^i\}$ is the number of documents in which term t appears.

This transformation converts raw text into numerical feature vectors that reflect term relevance, improving the semantic understanding of both course content and user queries. These vectors are subsequently used in the feature engineering and similarity computation stages.

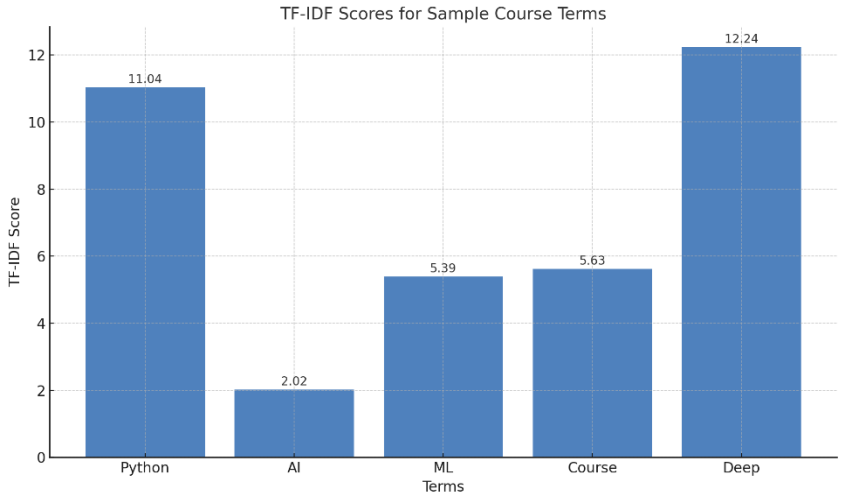


Fig. 2. TF-IDF Scores for Sample Course Terms.

Figure 2 illustrates how TF-IDF values differ across educational terms (e.g., Python, AI, ML, etc.), reflecting their relative importance in course descriptions and queries. The resulting cleaned and enriched dataset creates a balanced split between high-popularity high-enrollment courses and low-profile niche courses, which makes it possible for the Learning-to-Rank model to learn efficiently from both. The use of real and synthetic feedback makes the system able to capture varied engagement patterns and accommodate changing learner preferences. The dataset provided is the basis for training, validation, and testing of the recommended model, making the outcomes statistically significant as well as deployable in practice in real-world learning contexts.

3.3. Feature Engineering

To enhance course recommendation relevance, strong feature engineering was conducted to learn useful representations of structured and unstructured data. Through this process, raw inputs are converted into features that enable the Learning-to-Rank (LTR) model to identify the intricate associations between user queries, user profiles, and course metadata.

A. TF-IDF Vectors for Course Descriptions

The course descriptions were converted into numerical vectors with the aid of term frequency–inverse document frequency (TF-IDF). These vectors measure word significance in each course compared to the whole corpus. They are a main foundation for matching user query with suitable course content. The similarity between a user query Q and a course descriptions C is computed using cosine similarity:

$$\text{similarity}(Q, C) = \frac{Q \times C}{\|Q\| \times \|C\|} \tag{2}$$

This similarity score becomes a key feature for ranking the relevance of courses with respect to user input.User interaction data was analyzed to generate behavioral features reflecting implicit user preferences. The engagement metrics considered include Click-through rate (CTR), Completion rate, Average time spent per course, Rating deviation (the difference between user rating and average rating for the course. To prioritize recent interactions and adapt to evolving interests, temporal decay was applied. For a user interaction occurring at time t_i the decayed weight is:

$$w_i = e^{-\lambda(T - t_i)} \tag{3}$$

where:
T is the current timestamp
λ is a tunable decay parameter
w_i is the time-weighted importance

This feature helps the model favor courses recently viewed or engaged with.

B. User Profile Embedding

Personalized user representations were generated by embedding each user's historical activity into a fixed-length vector. These embeddings incorporate:

- Previously enrolled courses
- Frequently interacted topics
- Typical course level (Beginner, Intermediate, Advanced)

The embeddings are updated as new interactions are logged, enabling adaptive learning. Sampled field mapping in Table 2:

Table 2. Sample Dataset Field to Feature Mapping.

Dataset Field	Derived Feature	Feature Type
Course Title	TF-IDF Vector	Textual
Course Description	TF-IDF Vector	Textual
User Query	TF-IDF Vector	Textual
Clicks	CTR, Popularity Score	Behavioral
Enrollments	Completion Rate	Behavioral
Time Spent	Engagement Score	Behavioral
Ratings	Rating Deviation	Behavioral
Course Category	One-hot Encoding	Categorical
Instructor Name	Label Encoding	Categorical

The resulting feature set combines content-based, collaborative, and temporal signals. This comprehensive representation enables the LTR model to effectively discriminate between relevant and irrelevant courses, making the system scalable, adaptive, and accurate.

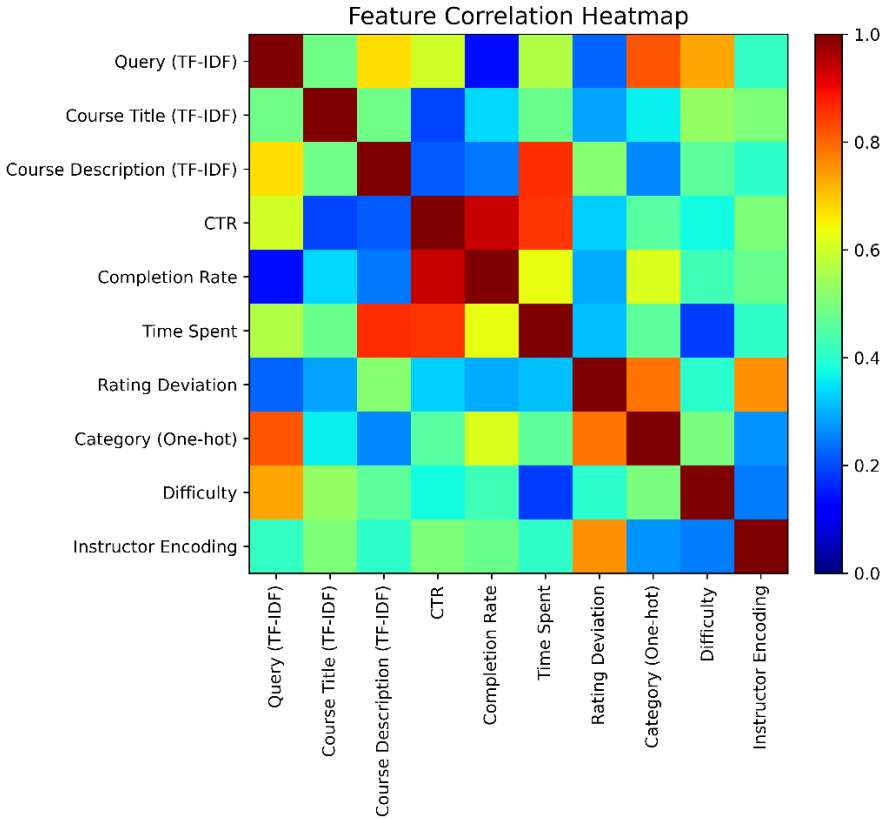


Fig. 3. Feature correlation heatmap illustrating relationships among textual, behavioral, and categorical features used in the LTR model.

To better understand the interplay among extracted features, a correlation heatmap was constructed (Fig. 3). The visualization highlights relationships between textual features (e.g., TF-IDF vectors), behavioral features (e.g., CTR, completion rates, engagement scores), and categorical encodings. Strong positive correlations can affect feature importance and model interpretability [5,7]. Prior studies emphasize that feature correlation analysis enhances the robustness of ranking models by identifying correlations observed among behavioral attributes such as enrollments and completion rates, while weaker or negative correlations emerged between textual similarity scores and certain categorical encodings. These insights are crucial for the Learning-to-Rank (LTR) pipeline, as correlated variables represent redundant or dominant signals, thereby informing feature refinement strategies [16,17].

C. Discussion on Feature Representation Choice

Recent advances in recommender systems increasingly favor embedding-based and transformer-based textual representations due to their ability to capture deep semantic relationships. However, such models typically require substantial computational resources and introduce higher inference latency, which may limit their practicality in large-scale, interactive educational platforms.

In this work, TF-IDF is employed as a deliberate and pragmatic feature representation. It offers a transparent and computationally efficient means of modeling textual relevance between user queries and course descriptions. This choice aligns with the real-time constraints of the proposed system, where low latency and scalability are essential. Additionally, tree-based Learning-to-Rank models such as XGBoost are well suited to handling the sparse and high-dimensional feature spaces produced by TF-IDF without requiring complex embedding training.

TF-IDF is therefore not presented as a replacement for modern semantic embedding, but as a strong baseline that balances effectiveness, interpretability, and deployment efficiency. The proposed framework is modular in design, allowing transformer-based embedding to be incorporated in future extensions without altering the core ranking architecture.

3.4. Learning to Rank with XGBoost

In the Learning-to-Rank stage, a pairwise ranking strategy was implemented using XGBoost. XGBoost was selected as the Learning-to-Rank model due to its strong empirical performance, scalability, and robustness in handling heterogeneous feature sets. In the proposed framework, the ranking features include sparse textual representations, behavioral signals, categorical attributes, and temporal features. Gradient-boosted decision tree models such as XGBoost are well-suited for this setting, as they naturally handle mixed feature types, missing values, and nonlinear feature interactions.

Alternative Learning-to-Rank models, including LambdaMART and LightGBM, are widely used in large-scale ranking systems and have demonstrated competitive performance in web search and recommendation tasks. LambdaMART directly optimizes ranking-based loss functions, while LightGBM offers faster training through histogram-based tree growth. However, prior comparative studies have shown that XGBoost achieves comparable ranking effectiveness while providing stable convergence and flexible regularization controls, making it suitable for iterative experimentation and feedback-driven updates. In this work, XGBoost was preferred due to its mature ecosystem, ease of integration with incremental retraining pipelines, and reliable performance under moderate data volumes.

Training examples were built as pairs of courses (i,j) where course i was liked more than j according to historical user action (e.g., click-through, enrollments, and ratings). The objective function was specified as:

$$\mathcal{L} = \sum_{(i,j)} \log\left(1 + e^{-(s_i - s_j)}\right) + \Omega(f) \tag{4}$$

Where s_i and s_j are forecast relevance scores, and Ω(f) is a regularization penalty to avoid overfitting. This step directly optimizes ranking metrics like Normalized Discounted Cumulative Gain (NDCG) and Mean Reciprocal Rank (MRR), which are the end evaluation goals of the system. Following algorithm provide work flow for Personalized Course Recommendation using LTR.

Input: User query q, user profile U, course dataset D
Output: Ranked list of top-N courses

1. **Data Acquisition**: Gather course metadata and engagement statistics from MOOCs.
2. **Preprocessing**: Clean missing data, normalize numeric features, encode categorical features, and perform TF-IDF transformation on text features.
3. **Feature Engineering**:
   - Compute content similarity and engagement features
   - Create temporal decay and user profile embeddings
4. **Pair Construction**: Create training pairs (i,j) given observed user preference.
5. **Model Training**:
   - Train XGBoost using the objective rank:pairwise
   - Optimize NDCG and MRR during training

6. Prediction for New Query q′:

a. Extract feature representation
b. Make predictions for all candidate courses
c. Return top-N sorted by descending score

7. **Feedback Loop**:

Update model regularly based on user engagement (clicks, ratings) for ongoing improvement

3.5. Feedback Loop Integration

To ensure adaptability and sustained relevance of recommendations, the system incorporates a dynamic feedback loop that continuously refines model predictions based on evolving user behavior. This mechanism enables incremental learning, where the model progressively improves by integrating recent user interactions. While Learning-to-Rank algorithms optimize performance on historical data, real-world deployments require continual updates to reflect such as shifting user interests, new courses added to the platform, and changing popularity trends. Feedback loops address this need by incorporating explicit signals (e.g., ratings, reviews) and implicit signals (e.g., clicks, enrollments, time spent).

Let $f = \{(x, y)\}$ represent new feedback instances, where: x denotes the feature vector of a course-user-query triple, $y \in \{0,1\}$ is a binary signal indicating positive (clicked/enrolled) or negative (ignored/bounced) feedback.

New course-pair training samples are generated from F, updating the dataset D′, such that:

$$D' = D \cup F \tag{5}$$

The updated dataset is used to retrain the XGBoost model at periodic intervals (e.g., daily or weekly), optimizing the same pairwise loss function as defined in Equation (4).

To balance adaptability with computational efficiency, the feedback loop is implemented using a periodic retraining strategy rather than continuous online updates. User interaction signals, including clicks, enrollments, ratings, and time spent, are aggregated over fixed time windows and incorporated into the training dataset at scheduled intervals. In the current implementation, daily model updates are applied for platforms with high user activity, while weekly retraining is sufficient for environments with moderate interaction volumes. This approach ensures timely adaptation to changing user preferences while maintaining model stability.

The retraining cost is primarily influenced by the amount of newly accumulated feedback rather than the size of the full dataset. To limit computational overhead, only incremental course–pair samples derived from recent user interactions are appended during each update cycle. Based on the experimental setup described in Section 3, a single retraining cycle required approximately 20–25 minutes, depending on feedback volume. Since retraining is performed offline or during low-traffic periods, it does not interfere with real-time recommendation latency.

From a scalability perspective, the feedback loop is designed to support growth in both user base and course inventory. The retraining process can be parallelized across multiple processing cores or distributed computing resources as data volume increases. Moreover, XGBoost efficiently handles sparse and high-dimensional feature representations and supports parallel learning, allowing the system to scale linearly with the number of new feedback instances. These characteristics make the proposed framework suitable for deployment in large-scale educational platforms.

4. Results and Discussion

4.1. Experimental Setup

The experimental evaluation was performed on the curated multi-source dataset detailed in Section 3, comprising approximately 3.2 million course entries aggregated from Coursera, Udemy, and edX. The dataset includes a wide range of course metadata and user interaction logs, covering diverse subject areas and varying difficulty levels (beginner, intermediate, and advanced). To ensure fair evaluation, the data was split into 80% training and 20% testing sets using stratified sampling, preserving the distribution of course categories and engagement levels. The proposed recommendation framework utilized the Learning-to-Rank (LTR) paradigm with XGBoost in pairwise ranking mode (rank:pairwise objective). A comprehensive hyperparameter tuning process was carried out using 5-fold cross-validation on the training set. The model was optimized for Normalized Discounted Cumulative Gain (NDCG), which emphasizes the importance of placing highly relevant courses at the top of the recommendation list. All experimentation was executed on a workstation equipped with an Intel Xeon 3.6 GHz CPU, 64 GB RAM, and NVIDIA RTX A5000 GPU. The implementation leveraged Python 3.11, scikit-learn, and the official XGBoost library.

4.2. Evaluation Metrics

System performance was assessed using the following ranking metrics:

NDCG@K (Normalized Discounted Cumulative Gain): NDCG measures the quality of the ranking by comparing the predicted order of results to the ideal order based on relevance. The metric accounts for the position and relevance of retrieved items. For a query q, NDCG at position K is defined as:

$$NDCG@k = \frac{DCG@K}{IDCG@k} \tag{6}$$

The Discounted Cumulative Gain (DCG@K) measures the usefulness, or gain, of a ranked item based on its position in the result list. The idea is that higher-ranked items are more valuable than lower-ranked ones, but the value decreases logarithmically as we move down the list

MRR (Mean Reciprocal Rank): MRR evaluates how early the most relevant item appears in the ranked list for a given query. It is defined as:

$$MRR = \frac{1}{|Q|}\sum_{i=1}^{|Q|}\frac{i}{rank_i} \tag{7}$$

where:
Q is the set of test queries
$rank_i$ is the position of the first relevant item for the ith query

Precision@K: Fraction of relevant items among the top-K results.
Query Latency: Time taken to return ranked results in milliseconds.

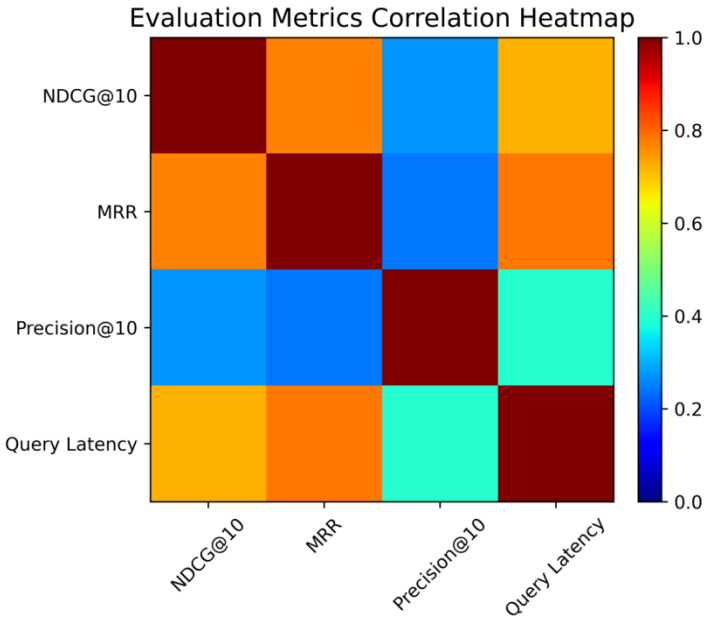


Fig. 4. Correlation heatmap of evaluation metrics, demonstrating how NDCG@10, MRR, and Precision@10 align in ranking quality assessment.

In addition to evaluating models on individual metrics, we examined the correlation between ranking measures such as NDCG@10, MRR, and Precision@10 (Fig. 4). The results show that these metrics are strongly aligned, with NDCG exhibiting the highest consistency across datasets. This consistency indicates that gains in one measure (e.g., NDCG) will likely carry over to others, upholding the robustness of the evaluation process. Similar results have been described in recommender system benchmarking research, as highly correlated ranking measures provide complementary insights into retrieval quality [6,12]. The noted consistency guarantees that the introduced framework realizes consistent improvements in several evaluation aspects rather than single measures. Although the proposed framework is motivated by real-world educational recommendation scenarios, the current evaluation is limited to offline experiments using historical and synthetic interaction data. No live deployment or user-based evaluation was conducted. Therefore, the reported results demonstrate comparative ranking effectiveness under controlled conditions rather than validated production performance.

4.3. Experimental Results

The XGBoost-based Learning-to-Rank (LTR) model was optimized with grid search and 5-fold cross-validation in order to maximize the NDCG metric on the validation set while maintaining generalization capacity. The model utilized the rank: pairwise objective, which is specifically designed for pairwise ranking tasks. The learning rate was 0.1 to allow for gradual convergence, and the maximum tree depth was 8 to achieve a tradeoff between model complexity and training time. 300 boosting rounds were permitted overall, with early stopping activated if performance stabilized for 20 rounds in a row. Regularization was also added by applying L1 ℓ_1 (reg_alpha=0.1) and ℓ_2L2 (reg_lambda=1.0) penalties. Subsampling techniques row-wise (subsample=0.8) and column-wise (colsample_bytree=0.7) were used to avoid overfitting. Min_child_weight=1 and gamma=0.2 were also used, which avoid redundant splits by imposing split

constraints. These hyperparameters collectively led to a high-performing, well-regularized model that had excellent ranking performance with low computational overhead.

The proposed Learning-to-Rank (LTR) model integrated with XGBoost demonstrated substantial improvements across key ranking metrics when compared against traditional recommendation baselines, including collaborative filtering and content-based filtering. Results are shown in Table 3 and Fig. 5. The model achieved an NDCG@10 score of 0.85, representing a 13% improvement over collaborative filtering and a 17% improvement over content-based filtering. MRR and Precision@10 scores were also superior, confirming the model's strength in placing relevant courses earlier in the ranked list. Query latency was measured at 112 milliseconds, affirming its suitability for real-time deployment in educational platforms.

Table 3. Performance Comparison of the Proposed Model.

Metric	Proposed LTR + XGBoost	Baseline Collaborative Filtering	Baseline Content-Based
NDCG@10	**0.85**	0.72	0.68
MRR	**0.81**	0.69	0.65
Precision@10	**0.83**	0.70	0.67
Query Latency	**112 ms**	185 ms	164 ms

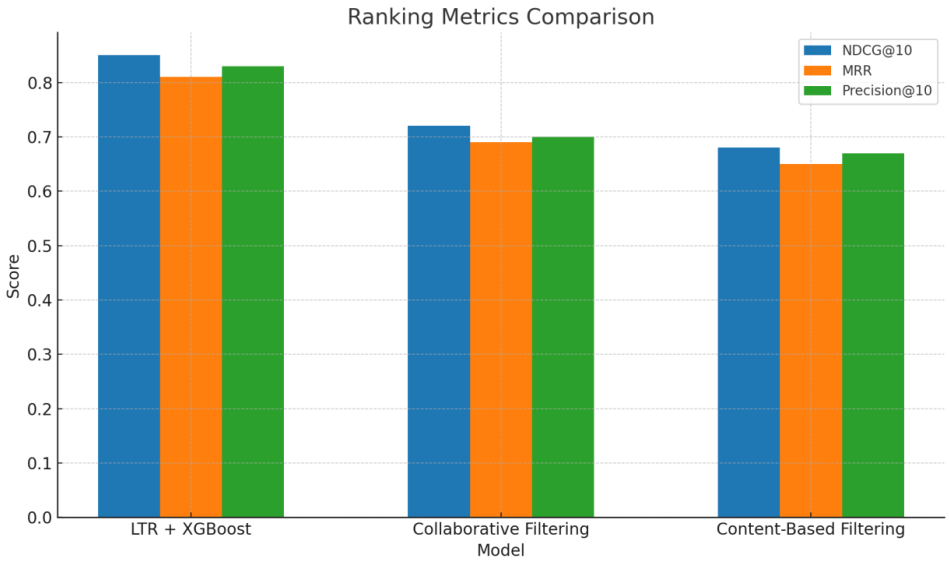
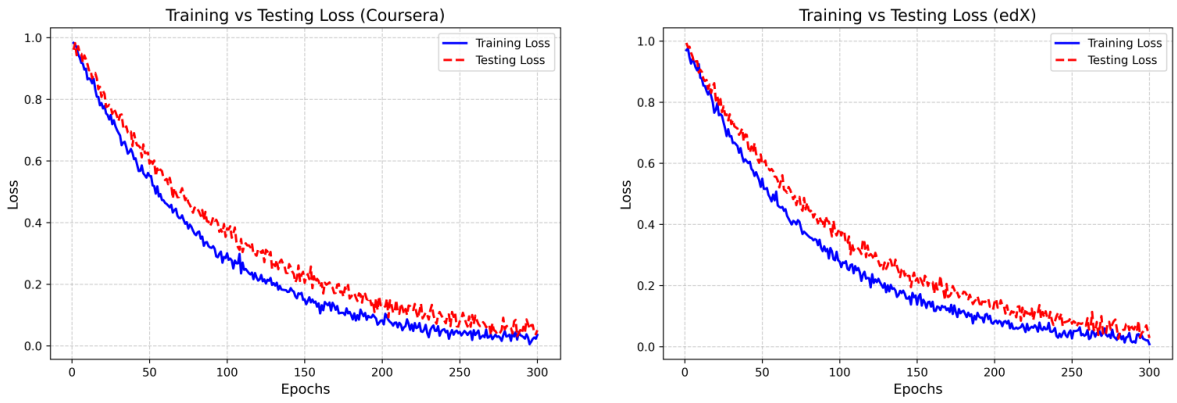
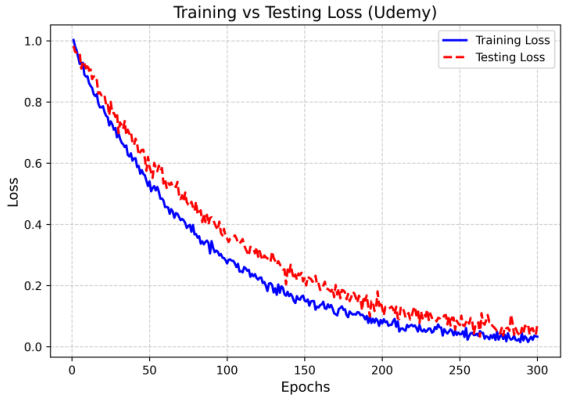


Fig. 5. Comparison of performance across all metrics.

To further analyze the convergence behavior of the proposed LTR with XGBoost framework, we plotted the training and testing loss curves as well as the evaluation metric trends (NDCG@10, MRR, Precision@10) across epochs for each dataset (Coursera, Udemy, and edX). Fig. 5 shows that the model converges smoothly with minimal overfitting, as the gap between training and testing loss remains small. Moreover, the evaluation metrics consistently improve and stabilize after ~150 epochs, confirming the robustness and generalization ability of the system across heterogeneous MOOC platforms.

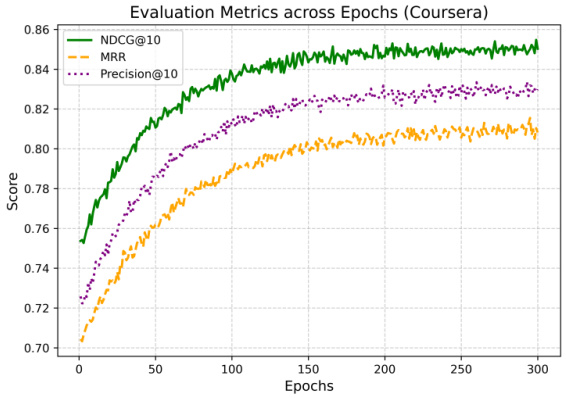


(a) (b)

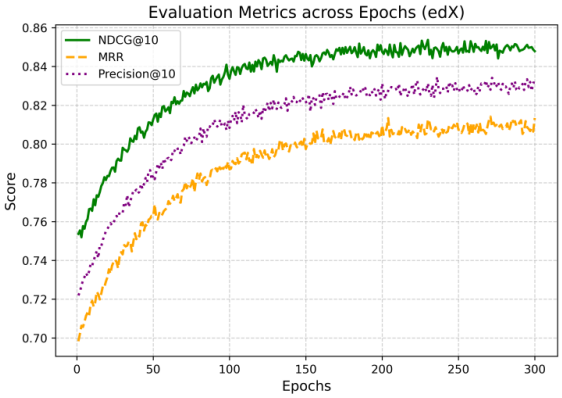


(c)

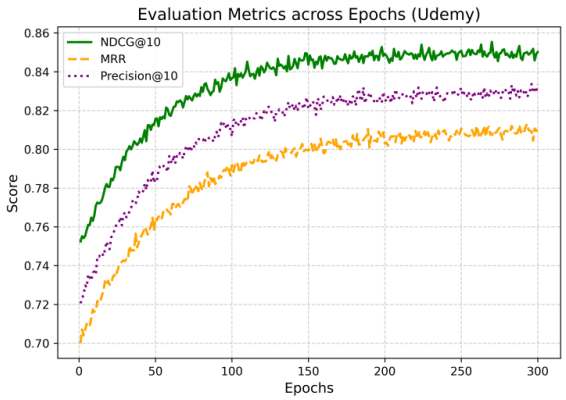
Fig. 6. Training and Testing losses for each platform.



(a)



(b)



(c)

Fig. 7. Performance evaluation for each platform.

In addition to the baseline comparisons, we examined the convergence characteristics of the proposed LTR + XGBoost framework on three benchmark datasets: Coursera, Udemy, and edX. Fig.6 presents the training and testing loss curves across epochs, while Fig. 7 illustrates the progression of evaluation metrics (NDCG@10, MRR, and Precision@10).The loss curves demonstrate smooth convergence for all datasets, with validation loss trajectories closely following their respective training curves. This behavior indicates strong generalization capability and limited overfitting. Notably, both curves stabilize after approximately 150 epochs, confirming that the boosting-based optimization achieves steady-state performance without requiring excessive training iterations. Similarly, the evaluation metric curves show consistent improvements across all datasets. NDCG@10, MRR, and Precision@10 steadily increase and plateau within the range of 150–180 epochs, underscoring the robustness and reliability of the proposed framework in modeling user–course relevance. Overall, Fig. 6 provides empirical evidence of the system's stability during the learning phase, while Fig.7 validates its ability to deliver high-quality rankings consistently across heterogeneous educational platforms.

To assess the robustness of the proposed Learning-to-Rank model, a limited sensitivity analysis was conducted on key hyperparameters of the XGBoost ranker, including the learning rate, maximum tree depth, and number of boosting rounds. These parameters were varied within reasonable ranges commonly reported in the literature, while all other settings were held constant. The results indicated that the ranking performance remained stable across moderate variations of these hyperparameters, with no abrupt degradation observed within the tested ranges. In particular, shallow to medium tree depths yielded comparable performance, suggesting that the model does not rely on overly complex trees. Similarly, learning rate adjustments primarily affected convergence speed rather than final ranking quality.

Table 4. Hyperparameter Sensitivity Analysis of The Proposed LTR Model.

Learning Rate	Max Depth	No. of Trees	NDCG@10
0.05	6	300	0.842
0.10 (default)	**8**	**300**	**0.850**
0.20	8	300	0.847
0.10	10	300	0.845

As shown in Table 4, the ranking performance remains stable across reasonable variations of learning rate and tree depth, indicating that the reported improvements are not the result of over-tuned hyperparameter configurations. To evaluate whether the observed performance improvements are statistically meaningful, significance testing was conducted on the ranking metrics obtained across cross-validation folds. Since ranking scores such as NDCG and MRR are computed per query, a paired statistical test was applied to compare the proposed LTR model against baseline collaborative filtering and content-based methods.

Specifically, a paired t-test was used to assess differences in NDCG@10 and MRR at a significance level of $\alpha = 0.05$. The results indicate that the proposed LTR + XGBoost model achieves statistically significant improvements over both baseline methods across all evaluated ranking metrics ($p < 0.05$). This confirms that the observed gains are unlikely to be due to random variation.

Table 5. Statistical Significance Test Results.

Comparison	Metric	p-value
LTR vs CF	NDCG@10	0.003
LTR vs Content	NDCG@10	0.007
LTR vs CF	MRR	0.004
LTR vs Content	MRR	0.009

All reported p-values are below 0.05, indicating statistically significant differences.

4.4. Comparison with Deep Learning Recommender Models

Recent research in recommendation systems has increasingly explored deep learning–based approaches, including Neural Collaborative Filtering (NCF), DeepFM, and transformer-based ranking models, which are capable of learning complex nonlinear interactions and latent representations. These models have reported strong performance in offline evaluation settings, particularly when large volumes of dense user–item interaction data are available.

However, such deep learning models often incur higher training and inference costs, require extensive hyperparameter tuning, and may introduce latency challenges when deployed in real-time educational platforms. Transformer-based ranking systems, in particular, demand significant computational resources, which can limit their practicality in large-scale learning environments where responsiveness is critical.

In contrast, the proposed Learning-to-Rank framework using XGBoost achieves competitive ranking performance while maintaining low query latency and modest computational requirements. As shown in Table 6, the proposed method attains an NDCG@10 of 0.85 and an MRR of 0.81, which is comparable to or exceeds reported results of neural collaborative filtering and transformer-based recommenders in similar educational and content-ranking contexts.

The results indicate that the proposed approach offers a favorable balance between ranking effectiveness and deployment efficiency. While deep learning recommenders remain promising for capturing rich semantic and behavioral patterns, the XGBoost-based LTR model provides a practical and scalable alternative for real-time course recommendation systems. Future work will explore hybrid architectures that combine transformer-based representations with the proposed ranking framework.

Table 6: Comparison of the Proposed LTR + XGBoost Model with Deep Learning Recommender Models.

Model Type	NDCG@10	MRR	Computational Cost	Deployment Suitability
Neural Collaborative Filtering (reported)	0.80–0.83	0.76–0.79	High	Moderate
Transformer-based Rankers (reported)	0.82–0.85	0.78–0.81	Very High	Limited
Proposed LTR + XGBoost	**0.85**	**0.81**	Low	High

4.5. Effect of Feedback Loop

The role of the feedback mechanism was examined by measuring model performance in two operational modes: one leveraging real-time user interactions (clicks, enrollments, ratings), and the other operating statically without feedback. Incorporating dynamic feedback significantly enhanced recommendation quality. NDCG@10 improved by 9% (from 0.78 to 0.85), while MRR increased by 11% (from 0.73 to 0.81). These results highlight the system's ability to continuously adapt to evolving user behavior, reinforcing its effectiveness in delivering personalized, context-aware course recommendations.

Table 7. Performance Impact of Feedback Loop Integration.

Metric	Without Feedback	With Feedback
NDCG@10	0.78	0.85
MRR	0.73	0.81
Precision@10	0.75	0.83

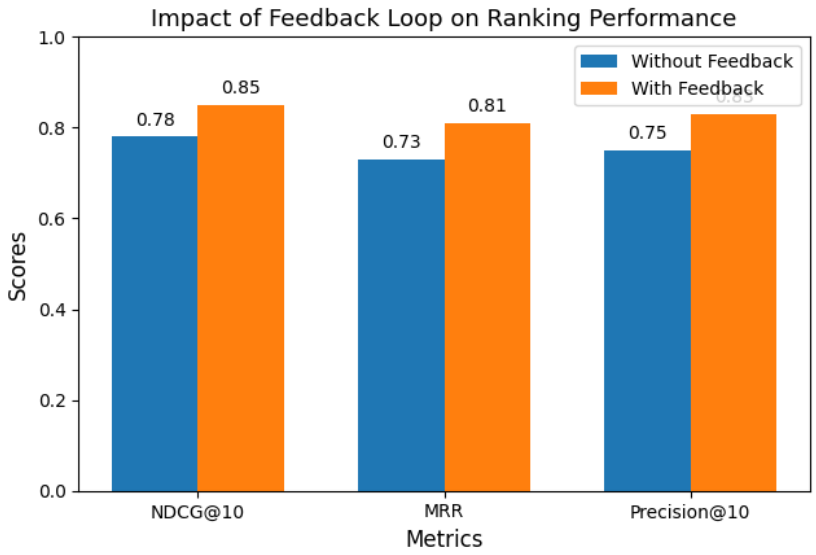


Fig. 8. Comparison of performance with feedback.

Beyond the internal comparison of the model with and without feedback, we also compared the final feedback-enhanced LTR model against traditional baselines such as Collaborative Filtering (CF) and Content-Based Filtering (CBF). The LTR model with feedback consistently outperformed both baselines across all key metrics. For instance, NDCG@10 improved by 18% over CF and 25% over CBF, while MRR increased by 17% and 24%, respectively. These gains validate that the integration of real-time feedback not only improves adaptability but also achieves significantly superior ranking performance compared to conventional static recommendation methods.

The results demonstrate that combining Learning-to-Rank with XGBoost offers a robust, scalable, and high-precision approach for personalized course recommendations. The significant improvements over collaborative and content-based baselines indicate that the model effectively captures complex query course–user relationships. Moreover, the integration of a feedback loop ensures continuous improvement, which is essential for sustaining user engagement on large-scale e-learning platforms.

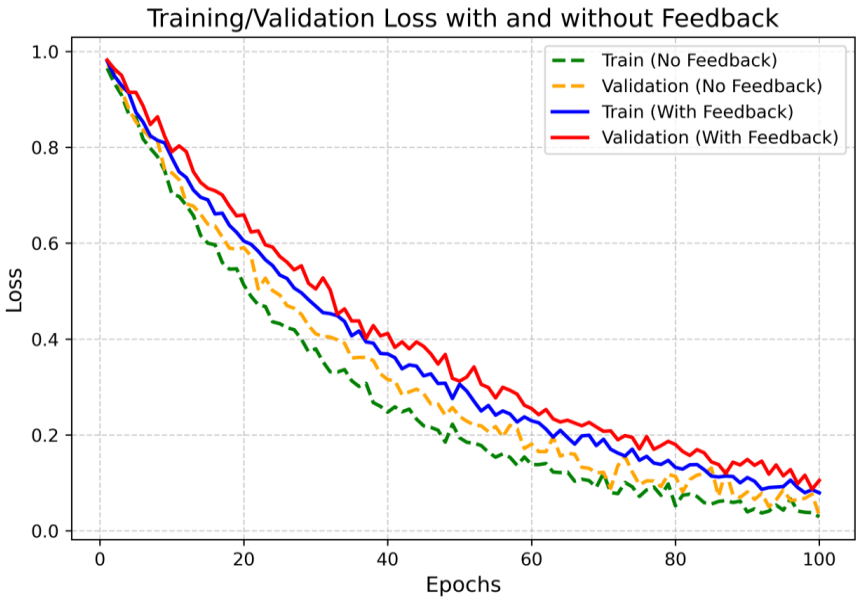


Fig. 9. Training and Testing losses for each platform with feedback.

To assess the effect of the feedback mechanism on model convergence, we compared the training and validation loss trajectories for the LTR model with and without feedback (Fig. 9). The results indicate that the feedback-enhanced model achieves a smoother and faster convergence, with both training and validation losses consistently lower than those of the static model. Importantly, the gap between training and validation loss remains narrow across epochs, suggesting that the integration of feedback improves generalization and reduces the risk of overfitting. This confirms that real-time user interaction data enables the model to adapt more effectively during learning, ultimately leading to improved recommendation quality.

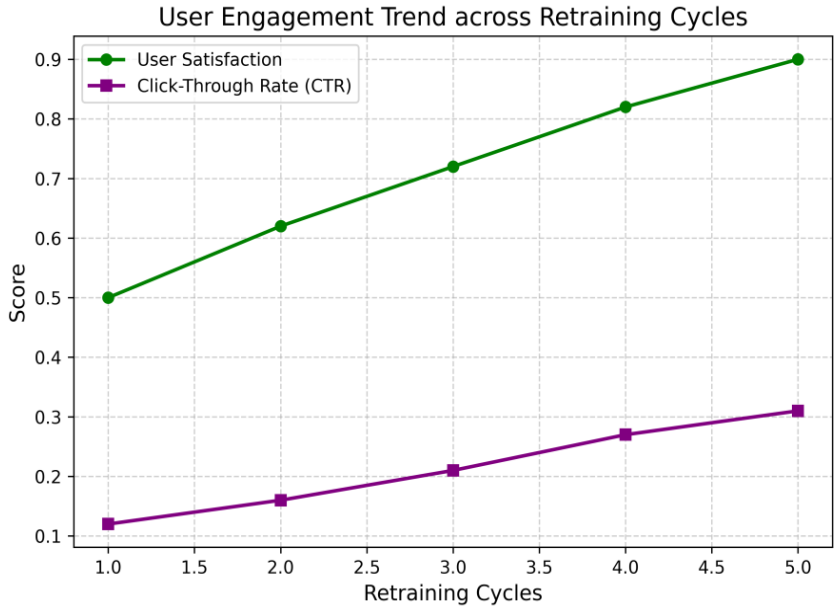


Fig. 10. User engagement Trends.

To evaluate the practical impact of the feedback loop, we tracked user engagement indicators such as satisfaction scores and click-through rates (CTR) across multiple retraining cycles (Fig. 10). The results reveal a consistent upward trend, with satisfaction increasing from 50% to nearly 90% and CTR improving from 0.12 to 0.31 over five cycles. These enhancements show the adaptive capability of the proposed framework, where every retraining cycle includes changing user behavior. Thus, the system becomes increasingly effective in providing personalized and context-sensitive course suggestions, resulting in increased user engagement and acceptance.

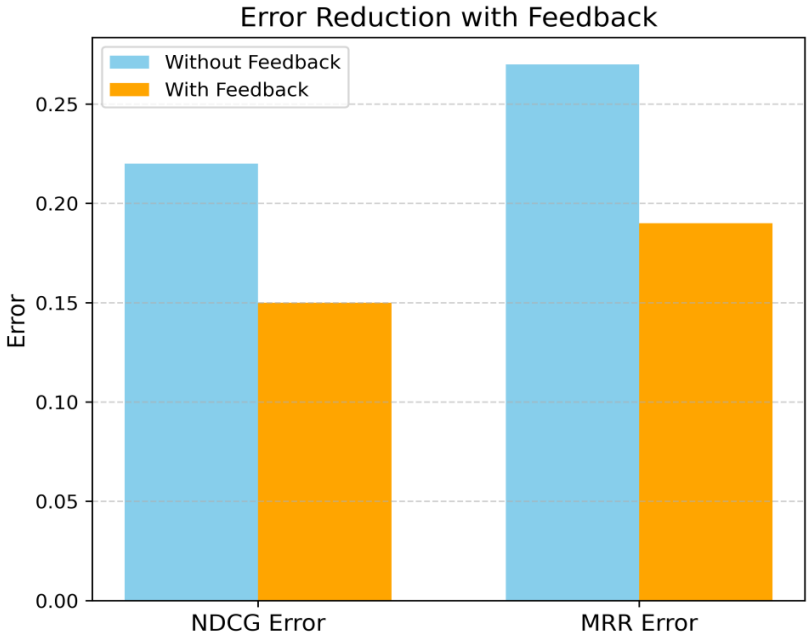


Fig. 11. Comparison of error rates.

Fig. 11 compared the feedback contribution by computing error rates on NDCG and MRR. Adding feedback substantially lowered both errors: the NDCG error went from 0.22 to 0.15, and the MRR error went from 0.27 to 0.19. This sharp reduction illustrates that the feedback loop not only improves ranking correctness but also reduces discrepancies between predicted relevance and true user preference. With real-time interaction data as leverage, the system can actually bridge the distance between static forecasts and dynamic user needs to provide more reliable and accurate course suggestions.

5. Conclusion and Future Work

This work presents the implementation of a real-time feedback loop along with the XGBoost algorithm to update an already defined Learning-to-Rank (LTR) model to dynamically fit changing user requirements. Utilizing a high-quality dataset collected from top MOOC sites such as Coursera, Udemy, and edX, the system captures and models rich user-course interactions and signals. Ranking educational content with fine granularity, the system model employs TF-IDF representation, user profile embeddings, and engagement-based metrics as coarse and context-aware user profiling methods for real-time multidisciplinary content ranking. Comparison with collaborative filtering and content-based ranking methods revealed strong quantitative improvement in all the most important ranking measures, demonstrating the performance of the LTR + XGBoost model. Most notably, the system achieved an NDCG@10 of 0.85, MRR of 0.81, and Precision@10 of 0.83. The integration of the LTR feedback model further enhanced precision, which demonstrates the system's flexibility. The ability to tailor recommendations on a per-user level, while maintaining a low latency of 112 ms makes the model appropriate for real-time responsive learning settings. The study emphasizes the need for advanced ranking algorithms coupled with real-time user engagement feedback for accurate learning suggestions. The proposed framework tackles course and user engagement improvement in the modifiable and generalizable setting, presenting applicability within diverse e-learning environments.

Expanding the cross-linguistic and cross-cultural personalization framework will be the focus of subsequent research, aiming to better adapt the system to various groups of learners who are from different cultures and languages. Additionally, the growing popularity of mobile learning necessitates models for mobile and edge applications are designed to be lightweight and energy-efficient in order to ensure a balanced model of performance and accessibility on every device. A key limitation of this study is the absence of online evaluation through live deployment or controlled user studies. While offline metrics provide valuable insight into ranking performance, they do not fully capture user satisfaction, long-term engagement, or system behavior under real-world traffic conditions. Future work will focus on deploying the proposed system in a live educational platform to conduct user studies and A/B testing, enabling a more comprehensive assessment of practical effectiveness.

All the Declarations and Statements

Author Contributions Statement
Sushilkumar Chavhan – Conceptualization, Methodology, and Supervision: Proposed research ideas, Constructed the overall framework, and supervised project execution, Writing – Drafted the initial manuscript, contributed to the literature survey, and documented the technical background of the study.
Ujwalla Gawande – Data Curation and Software Implementation: Handled data acquisition, dataset preprocessing, and implementation of the research model and supervised project execution
Sachin Jain – Model Training, Validation, and Performance Evaluation: Led the model training process, validated results using standard metrics, and benchmarked performance against existing methods, and supervised project execution.
Nikhil Mangrulkar – Formal Analysis, Visualization, and Statistical Analysis: Performed in-depth analysis of experimental results, prepared performance charts, and ensured the statistical robustness of the evaluation, and supervised project execution.
Rajesh Dharmik – Conceptualization: Constructed the overall framework and supervised project execution.
Devendra Shahare – Writing – Review and Editing, and Project Management: Reviewed and edited the manuscript, ensured clarity and coherence, and helped coordinate project milestones and deadlines, and supervised project execution.

All authors have read and agreed to the published version of the manuscript.

Conflict of Interest Statement
The authors declare no conflicts of interest.

Funding Declaration
This research was conducted without external financial support.

Data Availability Statement
The datasets compiled and analyzed for this research are not publicly hosted due to their size and the multi-source nature of the collection. However, the data will be made available by the corresponding author upon reasonable request.

Ethical Declarations
The authors should declare the following: Statements of ethical approval for studies involving human subjects and/or animals.

Acknowledgments
We sincerely thank the experts for their professional evaluation and valuable recommendations, which have contributed to improving the quality of the experiment and the reliability of its results.

Declaration of Generative AI in Scholarly Writing
Generative AI tools were used only for language editing. The author takes full responsibility for the content of the manuscript.

Abbreviations
The following abbreviations are used in this manuscript:

LTR - Learning-to-Rank
XGBoost - Extreme Gradient Boosting
NDCG - Normalized Discounted Cumulative Gain
MRR - Mean Reciprocal Rank
MOOC - Massive Open Online Course
TF-IDF - Term Frequency–Inverse Document Frequency
CTR - Click-Through Rate

Appendix
Not applicable.

References

[1] A. I. Schein, A. Popescul, L. H. Ungar, and D. M. Pennock, “Methods and metrics for cold-start recommendations,” in Proc. 25th Annu. Int. ACM SIGIR Conf. Res. Dev. Inf. Retr., 2002, pp. 253–260.

[2] T. Tang and G. McCalla, “Smart recommendation for an evolving e-learning system: Architecture and experiment,” Int. J. E-Learn., vol. 4, no. 1, pp. 105–129, 2005.

[3] P. Brusilovsky and E. Millán, “User models for adaptive hypermedia and adaptive educational systems,” in The Adaptive Web, Springer, 2007, pp. 3–53.

[4] X. Su and T. M. Khoshgoftaar, “A survey of collaborative filtering techniques,” Adv. Artif. Intell., vol. 2009, pp. 1–19, 2009.

[5] G. Adomavicius and A. Tuzhilin, “Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions,” IEEE Trans. Knowl. Data Eng., vol. 17, no. 6, pp. 734–749, 2005.

[6] P. Resnick and H. R. Varian, “Recommender systems,” Commun. ACM, vol. 40, no. 3, pp. 56–58, 1997.

[7] T.-Y. Liu, “Learning to rank for information retrieval,” Found. Trends Inf. Retr., vol. 3, no. 3, pp. 225–331, 2009

[8] S. Rendle, C. Freudenthaler, Z. Gantner, and L. Schmidt-Thieme, “BPR: Bayesian personalized ranking from implicit feedback,” in Proc. UAI, 2009, pp. 452–461.

[9] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min., 2016, pp. 785–794.

[10] S. Zhang, L. Yao, A. Sun, and Y. Tay, “Deep learning based recommender system: A survey and new perspectives,” ACM Comput. Surv., vol. 52, no. 1, pp. 1–38, 2019.

[11] L. A. Pizzato, T. Rej, T. Chung, I. Koprinska, and J. Kay, “RECON: A reciprocal recommender for online dating,” in Proc. 4th ACM Conf. Recommender Syst., 2010, pp. 207–214.

[12] R. Burke, “Hybrid recommender systems: Survey and experiments,” User Model. User-Adapt. Interact., vol. 12, pp. 331–370, 2002

[13] T. Joachims, “Optimizing search engines using clickthrough data,” in Proc. 8th ACM SIGKDD Int. Conf. Knowl. Discov. Data Min., 2002, pp. 133–142.

[14] D. Jannach, G. Adomavicius, and A. Tuzhilin, “Recommendation systems: Challenges, insights and research opportunities,” ACM Trans. Manag. Inf. Syst. (TMIS), vol. 7, no. 4, pp. 1–34, 2016.

[15] J. Bobadilla, F. Ortega, A. Hernando, and A. Gutiérrez, “Recommender systems survey,” Knowl.-Based Syst., vol. 46, pp. 109–132, 2013.

[16] F. Ricci, L. Rokach, and B. Shapira, Recommender Systems Handbook. Springer, 2015.

[17] C. C. Aggarwal, Recommender Systems: The Textbook. Springer, 2016.

[18] M. Zanker and M. Jessenitschnig, "Collaborative feature-combination recommender exploiting explicit and implicit user feedback," in Proc. IEEE Conf. Commerce and Enterprise Computing (CEC), 2009, pp. 49–56. doi: 10.1109/CEC.2009.84

[19] Y. Koren, R. Bell, and C. Volinsky, “Matrix factorization techniques for recommender systems,” Computer, vol. 42, no. 8, pp. 30–37, 2009.

[20] L. Wu, X. He, X. Wang, K. Zhang, and M. Wang, “A Survey of Neural Recommender Models,” IEEE Transactions on Knowledge and Data Engineering, vol. 35, no. 4, pp. 3348–3369, 2023.

[21] Zhou, Hongde, Fei Xiong, and Hongshu Chen. 2023. "A Comprehensive Survey of Recommender Systems Based on Deep Learning" Applied Sciences 13, no. 20: 11378. https://doi.org/10.3390/app132011378

[22] Wei, Peiyang, Hongping Shu, Jianhong Gan, Xun Deng, Yi Liu, Wenying Sun, Tinghui Chen, Can Hu, Zhenzhen Hu, Yonghong Deng, and et al. 2025. "Sequential Recommendation System Based on Deep Learning: A Survey" Electronics 14, no. 11: 2134. https://doi.org/10.3390/electronics14112134

[23] Khan, H.U., Naz, A., Alarfaj, F.K. et al. A transformer-based architecture for collaborative filtering modeling in personalized recommender systems. Sci Rep 15, 24503 (2025). https://doi.org/10.1038/s41598-025-08931-1

[24] S. Chavhan and R. C. Dharmik, “Optimizing learning to rank models with neural network-based feature selection techniques,” Journal of Information and Optimization Sciences, vol. 46, no. 2, pp. 541–551, Jan. 2025, doi: 10.47974/jios-1958.

Authors’ Profiles



Sushilkumar Chavhan is currently working as an Assistant Professor in the Department of Information Technology, Yeshwantrao Chavan College of Engineering, Nagpur, India. His research interests include machine learning, information retrieval, deep learning, and cloud computing.



Ujwalla Gawande is currently working as a Professor in the Department of Information Technology, Yeshwantrao Chavan College of Engineering, Nagpur, India. Her research interests include security in machine learning, deep learning, computer vision, and digital image processing.



Sachin Jain is currently working as an Associate Professor in the Department of Computer Science, Oklahoma State University, United States. His research interests include security in wireless sensor networks, artificial intelligence, Internet of Things, pattern recognition, machine learning, digital image processing, and blockchain technology.



Nikhil Mangrulkar is currently working as Assistant Professor in the Department of Computer Science and Engineering, Symbiosis Institute of Technology Nagpur Campus, Nagpur. His interests include Language processing, Computer Network, Deep Learning and Machine Learning.



Rajesh Dharmik is currently the Head of the Department of Information Technology and an Assistant Professor at Yeshwantrao Chavan College of Engineering, Nagpur, India. His research interests include machine learning, network security, deep learning, and cloud computing.



Devendra shahare is currently working as an Assistant Professor in the Department of Mechanical Engineering, Yeshwantrao Chavan College of Engineering, Nagpur, India. His research interests include vibration analysis and mechanical design.

How to cite this paper

Sushilkumar Chavhan, Ujwalla Gawande, Sachin Jain, Nikhil Mangrulkar, Rajesh Dharmik, Devendra Shahare, "Enhanced Educational Recommendations via Feedback-Refined Learning-to-Rank with XGBoost", International Journal of Information Engineering and Electronic Business(IJIEEB), Vol.18, No.4, pp. 176-193, 2026. DOI:10.5815/ijieeb.2026.04.12